Lab 3: CI/CD and ML Testing

GitHub Actions, Automated Testing, Data Validation, and Model Validation

Department of Computing
Jönköping University

Format: Optional and individual. No formal submission is required for this exercise. Approximate core duration: 2–3 hours; extensions: 1–2 additional hours. You may consult documentation and use AI assistance, but you should understand and be able to explain each check you implement.

1 Purpose

Unlike ordinary software, a working ML pipeline may accept invalid data or package a model with unacceptable predictive quality. You will complete a provided validation project, investigate deliberately broken data and a weak model, and configure GitHub Actions as a quality gate. The aim is not to optimize a medical predictor.

Learning outcomes

After the lab, you should be able to (1) distinguish code, data, and model checks; (2) identify schema, missing-value, range, uniqueness, and label problems; (3) evaluate a held-out classifier using recall, F1, and a calibration-related metric; (4) reject an inadequate model using explicit thresholds; (5) identify illustrative data drift and inspect subgroup performance; (6) configure a CI pipeline; and (7) explain cross-validation, calibration, reproducibility, and the difference between CI and deployment.

2 Dataset and supplied files

The project uses the Wisconsin Diagnostic Breast Cancer dataset, a *real* published dataset (569 observations). Measurements were extracted from digitized cell images. The project selects five numeric features and remaps the label to `malignant=1` and `benign=0`. This is **only a teaching exercise**, not a medically validated system. You must not use its predictions for clinical decisions.

- `data/train.csv`, `data/evaluation.csv`: fixed, stratified split. The evaluation split is for this exercise, not repeated model optimization.
- `data/train_faulty.csv`: intentionally modified training rows containing missing values, nonnumeric text, negative values, duplicate IDs, and an invalid target label.
- `data/evaluation_shifted.csv`: artificial feature shift, only for drift detection.
- `models/candidate.joblib`, `models/weak.joblib`: **already trained**; no training or file downloads needed. `train_models.py` shows how to reproduce them after validation works.
- `validation.py`, `model_checks.py`, `tests/`, and `.github/workflows/ml-ci.yml`: starter code with marked TODOs.
- `optional_extensions.py`, `serve_model.py`, `Dockerfile`: advanced activities, with guidance.

3 Part A: Explore and run the starter tests

Extract the archive, open a terminal in the project folder, and create a virtual environment:

```
python -m venv .venv
# macOS / Linux:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest tests/test_software.py -v
python -m pytest -v
```

The basic software tests should pass, but *several ML-specific tests should fail initially.* These failures are intentional: you must complete the TODOs, not remove or weaken the tests. Inspect `ml_config.py`, the datasets, and the test files. In `reflections.md`, record two examples of faults that a software smoke test would miss.

4 Part B: Data validation (D1–D6)

Open `validation.py`. The supplied `validate_data(df)` function returns a list of problems (an empty list means valid). One check is already complete: it detects missing required columns. **Fill in D1–D5:**

1. D1: reject missing feature or target values; hint: `df.isna()`.
2. D2: detect nonnumeric feature values without throwing an exception; hint: `pd.to_numeric(series, errors='coerce')` converts invalid strings to NaN.
3. D3: reject negative measurements; these five selected measurements are nonnegative by construction. Handle nonnumeric data safely before comparing.
4. D4: detect duplicate `sample_id` values; hint: `duplicated()`.
5. D5: accept only labels 0 and 1 in `malignant`; hint: `isin([0,1])`.

For example, `df['sample_id'].duplicated().any()` returns `True` if at least one ID is repeated. Each problem should append a useful message to `problems`, not silently drop rows. Investigate `data/train_faulty.csv` and write down which rules it violates (D6). Run:

```
python -m pytest tests/test_data.py -v
```

The clean dataset should pass; the faulty dataset should be rejected. **Reflect:** Is imputing a missing value the same thing as validating data? Why should an invalid dataset stop a training pipeline?

5 Part C: Model validation (M1–M3)

Open `model_checks.py`. The supplied function already calculates *recall* for malignant cases. Add:

1. M1: the F1-score using `f1_score(y, predictions)`.
2. M2: the Brier score using `brier_score_loss(y, model.predict_proba(X)[:, 1])`. A lower score is better; it evaluates predicted probabilities.
3. M3: a quality gate requiring F1 ≥ 0.85, recall ≥ 0.80, and Brier ≤ 0.15. These values are illustrative classroom criteria, not clinical standards.

Worked example: `recall_score(y, predictions)` measures $TP/(TP + FN)$. The F1-score is the harmonic mean $2PR/(P + R)$ of precision and recall. Examine why a majority-class predictor might have respectable accuracy but unacceptable F1 for the positive class. Run:

```
python -m pytest tests/test_model.py -v
python run_checks.py
```

You should see that `candidate.joblib` passes the model gate while `weak.joblib` fails it. No model training is required. In `reflections.md`, explain what each metric assesses and why one number is insufficient to claim a model is reliable.

6 Part D: Drift and subgroup performance (M4–M5)

M4: Implement `drift_detected()` using an absolute difference in the mean of `mean radius` between two datasets. Use > 2.0 as the exercise threshold. The supplied shifted dataset adds 8 to that feature. This is a *simple heuristic*, not a statistical drift test, and neither drift nor a distribution change proves model accuracy declined. **M5:** Implement `subgroup_recalls()` for samples below and at/above the evaluation set's median `mean radius`. Calculate recall separately for each subset. The provided `masks` dictionary already selects the rows. Inspect the number of positive cases per subgroup before interpreting the results. A subgroup with only a few positives has a very uncertain recall estimate. **Hints:** `df.loc[mask]` selects rows and `recall_score(subset['malignant'], model.predict(subset[FEATURES]), zero_division=0)` evaluates that slice.

7 Part E: Automate quality gates with GitHub Actions (C1–C4)

Create a GitHub repository for the project and inspect the starter workflow `.github/workflows/ml-ci.yml`. The checkout step, dependency installation, and software-test step are provided. Complete:

1. C1: add `actions/setup-python@v5` for Python 3.11.
2. C2: add a step running `python -m pytest tests/test_data.py -v`.
3. C3: add a step running `python -m pytest tests/test_model.py -v`.
4. C4: add a step running `python run_checks.py`.

Example step syntax (already used in the file):

```
- name: Software tests
  run: python -m pytest tests/test_software.py -v
```

Push the feature branch, open a pull request, inspect the *Actions/Checks* output, and review your own changes before merging. Try temporarily selecting `weak.joblib` in `run_checks.py`, observe failure, then restore the candidate. **Do not merge a deliberately failing version.** Official guide: https://docs.github.com/en/actions/tutorials/build-and-test-code/python.

8 Part F: Further ML validation and deployment (extensions)

These activities are recommended to cover the broader topic but are optional if you have limited time.

1. **Cross-validation (E1):** The provided `StratifiedKFold(3, shuffle=True, random_state=42)` creates three reproducible folds. Use `cross_val_score(..., scoring='f1')` and compare the fold scores. Fit the scaler *inside* the pipeline to avoid leakage. Reference: https://scikit-learn.org/stable/modules/cross_validation.html.
2. **Calibration (E2):** Use `calibration_curve(y, probabilities, n_bins=5)` to compare predicted probabilities with observed positive rates. Why is a well-ranked classifier not necessarily well-calibrated? Reference: https://scikit-learn.org/stable/modules/calibration.html.

3. **Reproducibility:** Run `python train_models.py` twice. Explain the purpose of fixed splits, random seeds, dependency versions, and saved model artifacts. Never load untrusted `joblib` files.

4. **Docker deployment (E3–E4):** `serve_model.py` provides a minimal Flask inference endpoint; complete the `Dockerfile`. Build with `docker build -t lab3-ml .` and run with `docker run -rm -p 5000:5000 lab3-ml`. Test `http://localhost:5000/health`. This introduces packaging, not clinical deployment or automated CD.

9 Conclusion

Write 150–250 words in `reflections.md`: explain a data-quality failure, a model-quality failure, what an automated gate prevents, why measured drift need not mean concept drift, and which additional evidence you would need before trusting an ML system in production. For this lab, passing tests means passing the *defined teaching checks*, not proving medical safety or clinical validity.